AINewsAgentic: A LangGraph-Based Agentic LLM Application for Web-Augmented Chat and Automated AI News Summarization

CSC 7644 - Applied LLM Development | Final Project Report | Sajol Md Saiful Islam

Git Link: https://github.com/MdSaifulIslamSajol/csc7644-final-project-Sajol

Abstract

Staying current with artificial intelligence news has become an exhausting daily task: relevant developments are scattered across hundreds of outlets, and traditional aggregators do not filter or summarize the content. This project, AINewsAgentic, is an agentic large language model (LLM) application built on LangGraph that addresses this problem through three composable use cases delivered from a single Streamlit interface: a basic chatbot, a chatbot that decides on its own when to invoke a Tavily web-search tool, and an AI-news pipeline that fetches recent articles and produces a dated, source-linked Markdown digest. The application uses Groq-hosted models (Llama 3.x, Llama 4-Scout, GPT-OSS-120B, and Qwen3-32B), the Tavily search API, LangChain, LangGraph, and Streamlit. An automated evaluation harness logged 80 runs across five models and three use cases. Llama-4-Scout produced the most reliable end-to-end behavior (100% tool-routing accuracy, 2.1-second AI-news latency, no errors), while smaller models showed factual errors on a domain-specific 'What is RAG?' prompt and over-eager tool calling. All evaluation runs stayed within the Groq free tier, with approximately 120,000 tokens consumed at near-zero cost. The project demonstrates that production-style agentic patterns - tool routing, multi-step graph pipelines, and structured logging -are achievable on free-tier infrastructure when paired with rigorous model selection.

1. Introduction and Background

AI practitioners, researchers, and decision-makers must monitor a rapidly moving field in which significant developments - model releases, regulatory actions, funding events, and academic results - are announced daily and distributed across hundreds of news sites, blogs, and social-media feeds. Manually scanning those sources consumes 30 to 60 minutes a day for many practitioners, and even then the coverage is incomplete. Existing tools partially mitigate the problem: RSS aggregators collect headlines, search engines retrieve articles on demand, and newsletters provide weekly digests. None of these solutions, however, perform intelligent filtering, deduplication, or summarization in a single, hands-free workflow.

An LLM-based solution is appropriate because the underlying tasks are exactly those at which large language models excel: identifying thematic relevance, summarizing prose, and rendering output in a structured format. When combined with a real-time web-search tool (to ground the model in current articles rather than its training data) and a small graph that chains fetch, summarize, and persist steps, the result is an autonomous workflow that produces a ready-to-read briefing. Beyond personal productivity, the underlying agentic pattern - a

controller LLM that orchestrates tool calls, retrieval, and post-processing - is the same pattern used for production assistants in finance, healthcare, and legal research. Building it end-to-end on free-tier infrastructure, with evaluation, demonstrates competence in modern LLM application engineering at low cost.

2. System Design and Implementation

2.1 Overall Architecture

AINewsAgentic is a Streamlit-fronted Python application organized into clearly separated modules: UI loading, LLM-client configuration, graph construction, individual graph nodes, tool wrappers, shared graph state, and an evaluation logger. At runtime, the user selects a Groq model and a use case from the sidebar; the app then constructs the appropriate LangGraph StateGraph and invokes it on the user's input.

Figure 1 summarizes the runtime data flow. The Streamlit layer captures input, the GroqLLM factory produces a configured ChatGroq client, the GraphBuilder selects one of three graphs, and the ResponseLogger writes a YAML-fronted Markdown record of every interaction for later analysis.

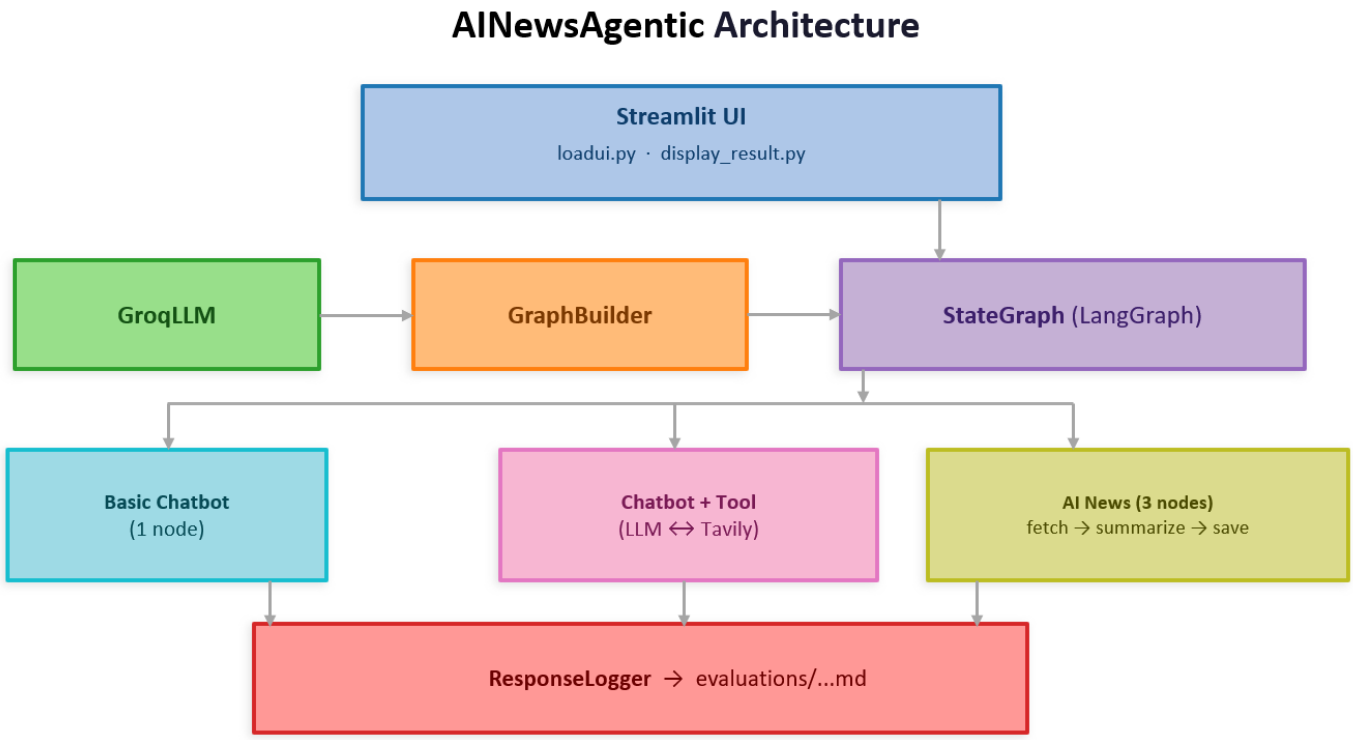

Figure 1. Runtime architecture. The Streamlit UI selects one of three LangGraph pipelines; every invocation is logged with latency, token usage, and tool metadata.

2.2 Three LangGraph Pipelines

The three use cases share state-graph plumbing but differ in structure:

- Basic Chatbot: a single node that calls the LLM on the user message and returns the reply. This serves as a sanity baseline and as a control for measuring tool-augmented latency.
- Chatbot With Web: a two-node graph (chatbot, tools) connected by LangGraph's tools_condition edge. The LLM decides at each step whether to call the Tavily search tool; the loop continues until the model returns a plain answer. This use case isolates the model's tool-routing judgment.
- AI News: a three-node pipeline (fetch_news, summarize_news, save_result) that issues a Tavily news query for the chosen time range, hands the article list to the LLM with a markdown system-prompt template, and writes the digest to disk. This is the primary deliverable that addresses the news-overload problem.

2.3 Data Sources, Models, and Prompt Design

All article content is retrieved live from the public web by the Tavily news endpoint, with the time_range parameter set to Daily, Weekly, or Monthly and the result count capped at 20. No static dataset is committed to the repository. Five Groq-hosted models were enabled for cross-model evaluation: llama-3.3-70b-versatile, llama-3.1-8b-instant, openai/gpt-oss-120b, meta-llama/llama-4-scout-17b-16e-instruct, and qwen/qwen3-32b. The summarization prompt is a ChatPromptTemplate that instructs the model to render each article in the strict format '### [Date]\n- [Summary](URL)', sorted newest first, with dates in the IST timezone. The chatbot-with-tool use case relies on LangChain's standard tool-binding interface; no additional system prompt is injected, so the model's native judgment drives routing.

2.4 Technical Stack and Justification

LangGraph provides the StateGraph abstraction needed for a multi-step pipeline with a shared TypedDict state, and is the natural choice for tool-using agents because tools_condition ships in its prebuilt module. LangChain supplies the Groq and Tavily integrations and message types. Streamlit was chosen over Flask or FastAPI because the UI is single-user and demonstrative; Streamlit collapses the front-end and back-end into one Python file. Groq Cloud was chosen as the LLM provider for its free-tier quota, low inference latency, and broad model selection. Tavily was chosen over generic search APIs because it exposes a news-specific topic filter.

2.5 Major Challenges

- Hidden state loss in LangGraph: the original State TypedDict declared only the messages key, so news_data and summary produced by intermediate nodes were silently dropped from graph.invoke() results. The fix was to add news_data, summary, usage_metadata, and filename as NotRequired fields in state.py.
- Free-tier token-per-minute caps: a 20-article news prompt exceeded Qwen3-32B's 6,000 TPM limit, and gpt-oss-120b hit caps during back-to-back batch runs. The batch runner adds 1- to 2-second sleeps between calls to stay under quota.
- UI feedback during long calls: a 6-second AI-news call feels broken without visual feedback. The chat-input border was given three CSS-driven states (idle/processing/error) so the user always knows the system status.

3. Results and Evaluation

An automated harness (batch_run.py) executed 80 logged runs covering 5 models x 3 use cases, with 5 prompts per chatbot case, a 5-trigger / 5-no-trigger split for the tool-routing case, and the Weekly frequency for AI News. Every response was written to evaluations/<usecase>/<model>/<timestamp>.md with YAML frontmatter capturing latency, token usage, and (for the tool case) routing correctness. The aggregation script evaluate.py walks that tree and produces both summary tables and a CSV roll-up.

Repository Organization

```
AINEWSAgentic/
├── app.py                      # Streamlit entrypoint (calls main.load_langgraph_agenticai_app)
├── batch_run.py                # Runs every model × prompt × use case combo
├── evaluate.py                 # Aggregates evaluations/ into summary tables + CSV
├── requirements.txt
├── .env.example                # Template — copy to .env and fill in keys
├── AINews/                     # Generated AI-news digests (daily/weekly/monthly)
├── evaluations/                # Per-run logs written by ResponseLogger (gitignored)
├── EVALUATION_REPORT.md        # Written-up findings from the batch runs
├── evaluation_summary.csv      # Aggregated metrics produced by evaluate.py
└── src/
    └── langgraphagenticai/
        ├── main.py             # Streamlit app loop and status-border styling
        ├── LLMS/groqllm.py     # Groq model factory
        ├── graph/graph_builder.py      # Builds one of three LangGraph graphs
        ├── nodes/
        │   ├── basic_chatbot_node.py
        │   ├── chatbot_with_Tool_node.py
        │   └── ai_news_node.py         # fetch_news / summarize_news / save_result
        ├── tools/search_tool.py        # Tavily search tool + ToolNode factory
        ├── state/state.py              # TypedDict graph state
        ├── ui/
        │   ├── uiconfigfile.ini        # Page title, model list, use-case list
        │   ├── uiconfigfile.py
        │   └── streamlitui/            # loadui.py, display_result.py
        └── evaluation/logger.py        # ResponseLogger + token-usage extractor
```

3.1 Functional Completeness

Model	Basic Chatbot	Chatbot With Web	AI News	Verdict
llama-3.3-70b-versatile	5/5	10/10	1/1	Full
llama-3.1-8b-instant	5/5	8/10	1/1	Partial
openai/gpt-oss-120b	5/5	8/10	1/1	Partial
meta-llama/llama-4-scout	5/5	10/10	1/1	Full
qwen/qwen3-32b	5/5	9/10	0/1	Fails AI News

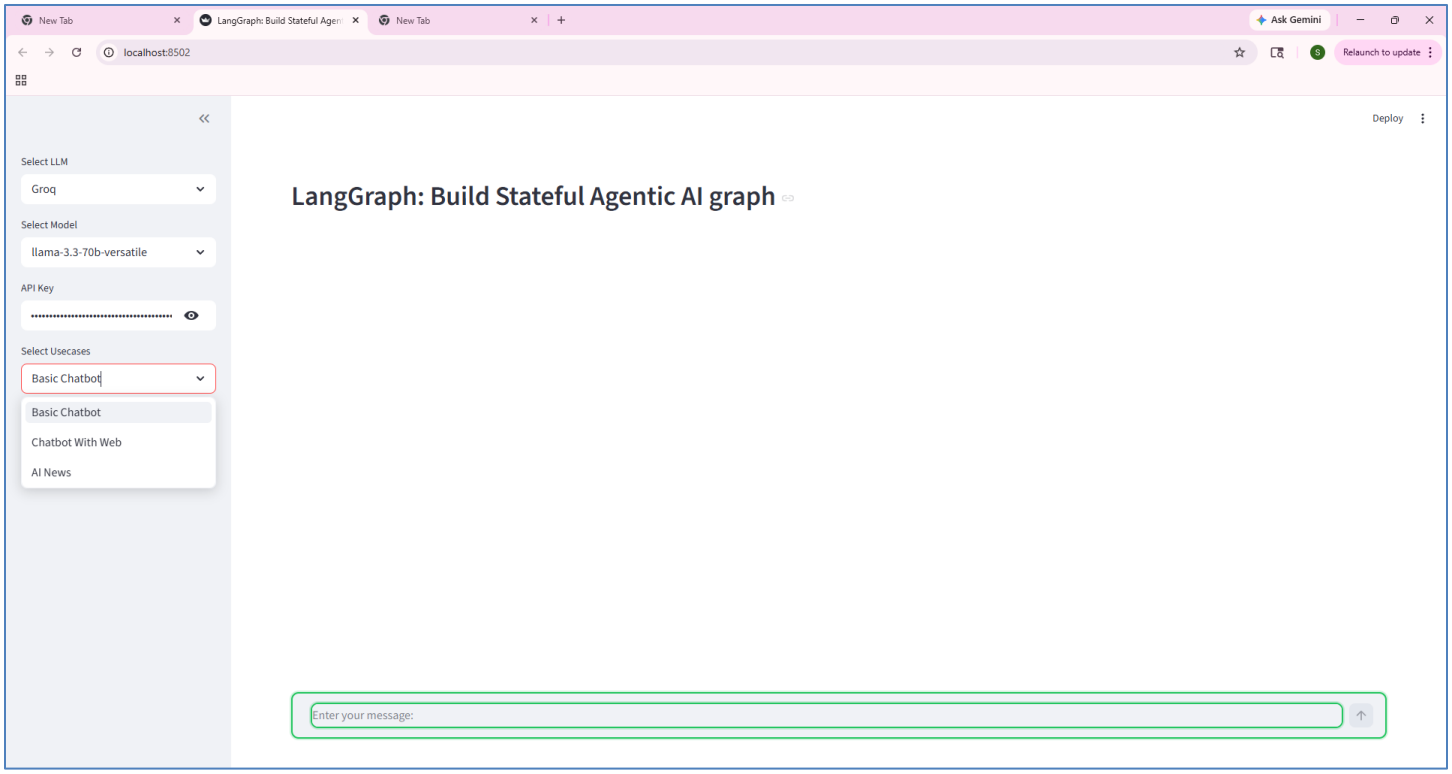

Figure 2. Streamlit application UI. Sidebar controls drive provider, model, API-key, and use-case selection; the chat input uses a color-coded border to surface system state (green=idle, yellow=processing, red=error).

Figure 2 captures the Streamlit entry screen. The sidebar exposes the three configuration controls a user touches at runtime - the LLM provider, the specific Groq model, and the API key - followed by the use-case selector with all three pipelines visible. The chat-input border at the bottom of the canvas is rendered green to signal the idle state; it transitions to yellow while the LLM is processing and to red on error, as described in section 2.5.

3.2 Latency

Model	Basic Chat (s)	Web Chat (s)	AI News Weekly (s)
llama-3.3-70b-versatile	1.3	2.4	3.2
llama-3.1-8b-instant	0.9	9.9	2.3
openai/gpt-oss-120b	2.1	21.4	6.9
meta-llama/llama-4-scout	1.1	1.9	2.1
qwen/qwen3-32b	3.3	13.0	n/a

All four successful models cleared the proposed AI-News latency targets (Daily <= 30s, Weekly <= 45s) by a wide margin. Total token consumption across 80 runs was approximately 120,000 tokens; under Groq free-tier rates the actual cost was $0.00, and a worst-case estimate at on-demand pricing is roughly $0.10. Free-tier compliance was therefore verified.

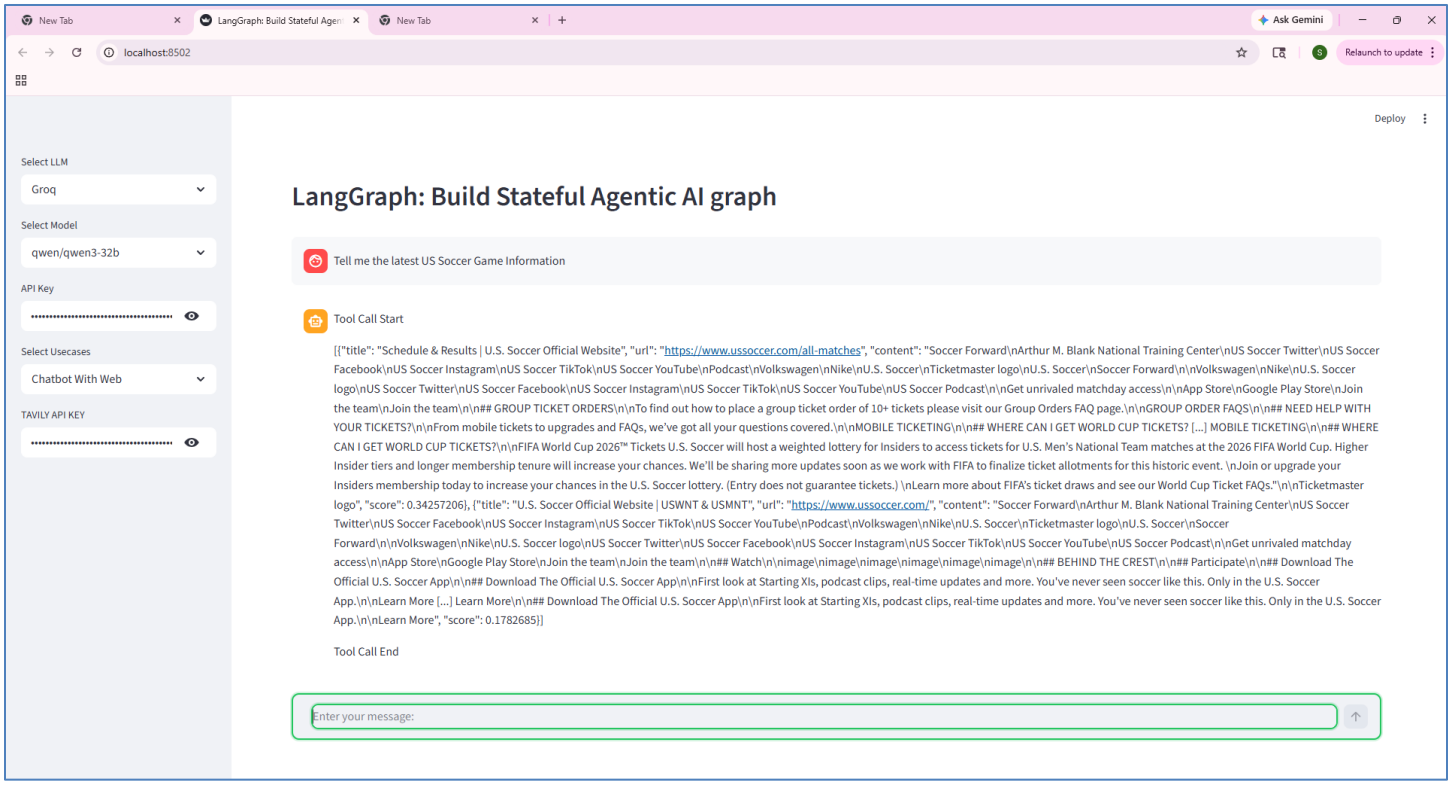

Figure 3. Chatbot With Web use case. The agent autonomously decides to invoke the Tavily search tool; the raw tool output is rendered between explicit start/end markers before the model's final answer is produced.

Figure 3 shows the Chatbot With Web use case in action. The user asked for live information that the model could not answer from its weights alone, so the LangGraph tools_condition edge routed the message to the Tavily search node. The 'Tool Call Start' / 'Tool Call End' delimiters are emitted by display_result.py to make the tool invocation explicit to the user; the JSON-shaped payload between them is the raw search result the model then summarizes back into the conversation.

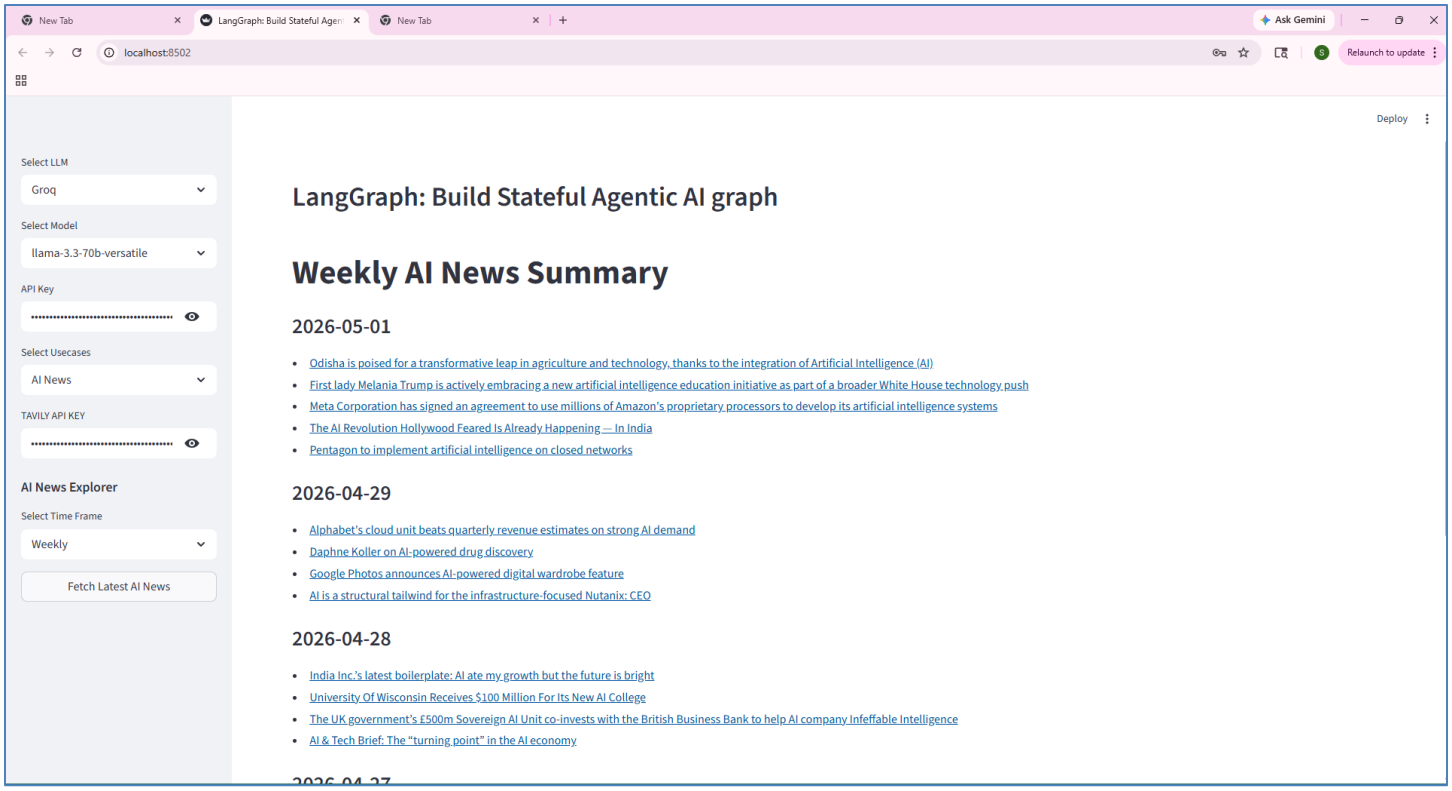

Figure 4. AI-News pipeline output. The Weekly digest groups twenty Tavily-sourced articles by IST date and links each item to its original source. The same view is also persisted as ./AINews/weekly_summary.md.

The AI-News output in Figure 4 was generated by the three-node pipeline (fetch_news, summarize_news, save_result) running with the Weekly time range. The system pulled twenty Tavily news articles, passed them to the selected Groq model with the markdown system prompt, and rendered the resulting digest grouped by date in newest-first order. Each line is a live, clickable source URL, which is the exact contract specified in the summarize_news prompt template.

3.3 Tool-Routing Accuracy

Model	Correct routes	Accuracy
meta-llama/llama-4-scout	10/10	100%
openai/gpt-oss-120b	8/8	100%
qwen/qwen3-32b	9/9	100%
llama-3.3-70b-versatile	9/10	90%
llama-3.1-8b-instant	5/8	62.5%

The 8B Llama was over-eager: it called Tavily for textbook questions such as 'Explain how recursion works' and 'Translate good morning to French.' This wastes both latency and tokens and is the dominant signal against using small models as tool controllers.

3.4 Qualitative Findings

Manual review of representative outputs surfaced two qualitative issues invisible to the latency and routing metrics. First, both Llama 3.x models answered 'What is RAG?' with the project-management meaning ('Red, Amber, Green') rather than Retrieval-Augmented Generation, despite the obvious AI/ML context - a production-blocker for a developer audience. Second, the same Llama models rendered the AI-news output as a list of headline links rather than the requested 'concise sentence summaries,' violating the system prompt. Llama-4-Scout and GPT-OSS-120B followed the prompt faithfully and produced well-structured markdown with concrete facts and figures. Qwen3-32B answered the RAG question correctly but leaked an internal '<think>' reasoning block into the visible response, a UX regression rather than a content error.

4. Discussion

What worked well: the LangGraph StateGraph abstraction made it straightforward to express three structurally different pipelines while reusing the same UI, LLM client, and logging infrastructure. The decision to log every response as a Markdown file with YAML frontmatter, rather than to a database, made downstream analysis trivial - evaluate.py is fewer than 150 lines and produces all reported tables. The batch harness exposed cross-model differences that would not have been visible from a handful of manual prompts; in particular, the RAG-acronym failure and the news-summary format failure were both surfaced by automated runs that a human would not have thought to repeat across all five models.

What did not work as expected: model size and brand recognition turned out to be poor proxies for fitness. The ostensibly weaker llama-4-scout-17b outperformed the larger llama-3.3-70b on the most production-relevant tasks (RAG knowledge, news-prompt adherence, tool routing). The largest model in the test, gpt-oss-120b, produced the highest-quality output but was throttled into impracticality on the free tier. The smallest model, llama-3.1-8b-instant, exhibited both the tool-overcalling and the RAG hallucination, and is not recommended for any path in this application.

Limitations: only the Weekly frequency was tested for the AI-News pipeline to keep the evaluation within Tavily's free quota; Daily and Monthly were not benchmarked. The tool-routing test set is small (10 prompts) and English-only. Output quality was scored by a single human reviewer rather than a panel, so qualitative judgments may be subjective. Finally, the news pipeline does not deduplicate articles or score source reliability, so two outlets covering the same story will both appear in the digest.

Future work: an LLM-as-judge evaluation pass over the existing logs would scale qualitative scoring; a small embedding-based deduplication step before summarization would tighten the news output; and replacing the static system prompt with a retrieval step over a curated AI-glossary would likely fix the RAG-acronym failure on the smaller models. A longer-horizon extension would adapt the same three-node template to other domains (security advisories, healthcare regulatory news) by swapping the search query and the system prompt.

5. Conclusion

AINewsAgentic delivers a working, evaluated agentic LLM application that addresses a concrete problem - AI-news overload - with three integrated use cases: a baseline chat, a tool-augmented chat, and an autonomous news pipeline. A structured evaluation across five models and 80 runs demonstrated that meta-llama/llama-4-scout-17b is the best all-round model for this stack, that openai/gpt-oss-120b produces the highest-quality content but is rate-limited on the free tier, and that the smallest Llama is unsuitable for tool routing. Beyond the headline results, the project produced reusable infrastructure: a YAML-fronted ResponseLogger, a batch-run harness, and an aggregation script that together turn ad-hoc demonstrations into reproducible experiments. The personal lessons were that LangGraph state must be declared explicitly to survive graph.invoke, that free-tier TPM caps shape model choice as much as quality does, and that automated cross-model evaluation surfaces failure modes that no single demo would expose. These lessons map directly onto the broader practice of applied LLM development, where careful evaluation, tight feedback loops, and a sober view of model limitations matter more than scale alone.

References

LangChain. (2024). LangGraph documentation. https://langchain-ai.github.io/langgraph/

LangChain. (2024). LangChain Python documentation. https://python.langchain.com/

LangChain. (2024). ChatGroq integration. https://python.langchain.com/docs/integrations/chat/groq/

Tavily. (2024). Tavily Search API documentation. https://docs.tavily.com/

Groq. (2024). Groq Cloud console and pricing. https://console.groq.com/

Streamlit. (2024). Build chat and LLM applications. https://docs.streamlit.io/develop/tutorials/chat-and-llm-apps

Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.